Verifying ABA with Leslie

Gaspard Reghem

June 1, 2026

Abstract

Our main objective is the verification of an implementation of Asynchronous Byzantine Agreement. The proof of correctness consist in building a simulation relation between the implementation and an implementation runnning in an idealised setting. The simulation ensures that the correctness of the idealised implementation implies the correctness of the original one. However, this implementation is complex, thus, building the simulation in one go is intractable. Fortunately, the implementation is designed as the interlocking of multiple sub-protocols (GBCA, WCC, RSD, Gather, AVSS, BRB). This architecture allows us to decompose the construction of the overall simulation into smaller steps. Each implementation of sub-protocols will be simulated by an idealised version. Thus, when building the simulation for a protocol using sub-protocols, the sub-implementations will be replaced by their idealised versions; making the construction easier.

1 Preliminaries

We introduce the semantic framework used to model protocols and their simulations: probabilistic labelled transition systems, probabilistic forward simulations, and the bridge to the non-probabilistic setting.

Probabilistic Labelled Transition System

Definition 1. A *labelled transition system* (LTS) on a state space S and label space L is a pair $(\mathit{init}, \mathit{step})$ with $\mathit{init} \subseteq S$ and $\mathit{step} \subseteq S \times L \times S$. We write $s \xrightarrow{l} s'$ for $(s, l, s') \in \mathit{step}$.

Definition 2. A *probabilistic labelled transition system* (PLTS) on S and L is a pair $(\mathit{init}, \mathit{step})$ with $\mathit{init} \subseteq S$ and $\mathit{step} \subseteq S \times L \times \mathsf{PMF}(S)$, where $\mathsf{PMF}(S)$ is the set of probability mass functions on S. We write $s \xrightarrow{l} \mu$ for a transition to distribution μ.

Definition 3. A *labelling* on L designates a subset of *internal* (silent / τ) labels and a distinguished internal label τ. Non-internal labels are called *external*.

Definition 4. A state s is *reachable* in a PLTS if there is a finite sequence $s_0, s_1, \ldots, s_n = s$ with $s_0 \in \mathit{init}$ and, for each $k < n$, a transition $s_k \xrightarrow{l_k} \mu_k$ with $s_{k+1} \in \mathrm{supp}(\mu_k)$.

Probabilistic forward Simulation A probabilistic forward simulation relates each concrete state to a *distribution* over abstract states. Matching uses the natural lifting of such a relation to distributions.

Definition 5. Given $R : S_1 \to \mathsf{PMF}(S_2) \to \mathsf{Prop}$ and distributions $\mu : \mathsf{PMF}(S_1)$, $\nu : \mathsf{PMF}(S_2)$, the *distribution lifting* $\mathsf{DistLiftR}(R, \mu, \nu)$ holds when there exists a choice function $f : S_1 \to \mathsf{PMF}(S_2)$ with $R(p, f(p))$ for every $p \in \mathrm{supp}(\mu)$ and $\nu = \mu \gg\!= f$.

Definition 6. A *weak transition* $\mu \overset{l}{\Rightarrow} \mu'$ in a PLTS consists of: a finite chain of internal hyper-transitions from μ, followed by a single hyper-transition with label l, followed by another chain of internal hyper-transitions ending in μ'. A *hyper-transition* $\mu \xrightarrow{l} \mu'$ lifts the single-state *step* relation pointwise: for each $q \in \mathrm{supp}(\mu)$ there is $q \xrightarrow{l} f(q)$, and $\mu' = \mu \gg\!= f$.

Definition 7. A *probabilistic forward simulation* from a concrete PLTS C (with labelling λ_1) to an abstract PLTS A (with labelling λ_2) consists of:

- a relation $R : S_1 \to \mathsf{PMF}(S_2) \to \mathsf{Prop}$,
- a label map $\mathit{lab} : L_1 \to L_2$,
- *(init)* for every concrete initial state s_1, a distribution ν supported on init_A with $R(s_1, \nu)$,
- *(step)* for every reachable s_1 with $R(s_1, \nu)$ and concrete step $s_1 \xrightarrow{l_1} \mu_1$, an abstract weak transition $\nu \overset{\hat{l}}{\Rightarrow} \nu'$ — internal if l_1 is internal, otherwise labelled $\mathit{lab}(l_1)$ — with $\mathsf{DistLiftR}(R, \mu_1, \nu')$.

We say the abstract A *simulates* the concrete C when such an R exists.

Theorem 8. *Let R be a probabilistic forward simulation from C to A. If a state predicate P on S_1 holds on every s_1 that is related to some abstract distribution by R, then P holds on every reachable concrete state, along every valid execution of C.*

Theorem 9. *Let R be a probabilistic forward simulation from C to A. For every concrete adversary strategy σ_1 and initial state s_1, there exist an abstract strategy σ_2 and initial state s_2 such that the external-trace distribution of A under σ_2 from s_2 equals the external-trace distribution of C under σ_1 from s_1, pushed forward through lab.*

Equivalence in the Non-Probabilistic Setting Several sub-protocols (e.g. BRB) are purely non-probabilistic. For these, the simulation obligation reduces to a standard relational forward simulation between plain LTS. The two settings are reconciled by a Dirac embedding fromLTS and an unfolding toLTS.

Definition 10. Given an LTS $\mathcal{L}$, the PLTS $\mathsf{fromLTS}(\mathcal{L})$ has the same initial states as $\mathcal{L}$ and a transition $s \xrightarrow{l} \delta_{s'}$ (the Dirac distribution at s') iff $s \xrightarrow{l} s'$ holds in $\mathcal{L}$.

Definition 11. Given a PLTS $\mathcal{P}$, the LTS $\mathsf{toLTS}(\mathcal{P})$ has the same initial states and a transition $s \xrightarrow{l} s'$ iff there exists μ with $s \xrightarrow{l} \mu$ in $\mathcal{P}$ and $s' \in \mathrm{supp}(\mu)$.

Theorem 12. $\mathsf{toLTS}(\mathsf{fromLTS}(\mathcal{L})) = \mathcal{L}$ *for every LTS* $\mathcal{L}$.

Definition 13. A *forward simulation* from a concrete LTS $\mathcal{C}$ to an abstract LTS $\mathcal{A}$ is a relation $R : S_1 \to S_2 \to \mathsf{Prop}$ with a label map such that: initial states are related; every internal concrete step from a reachable pair is matched by zero or more internal abstract steps; every external step $s_1 \xrightarrow{l_1} s_1'$ is matched by zero-or-more internal abstract steps, one external abstract step with label $lab(l_1)$, and zero-or-more internal abstract steps — ending in a state related to s_1'.

Definition 14. Given a forward simulation R between LTS $\mathcal{C}$ and $\mathcal{A}$, the construction ForwardSim_to_ProbForwardSi produces a probabilistic forward simulation between $\mathsf{fromLTS}(\mathcal{C})$ and $\mathsf{fromLTS}(\mathcal{A})$ with relation $R'(s_1, \nu) := \exists s_2,\ \nu = \delta_{s_2} \wedge R(s_1, s_2)$.

Definition 15. Conversely, the construction ProbForwardSim_to_ForwardSim takes a probabilistic forward simulation between $\mathsf{fromLTS}(\mathcal{C})$ and $\mathsf{fromLTS}(\mathcal{A})$ whose relation only relates concrete states to Dirac distributions, and returns a forward simulation between $\mathcal{C}$ and $\mathcal{A}$.

In particular, to prove that the ideal version of a non-probabilistic sub-protocol simulates its concrete implementation, it suffices to exhibit a forward simulation between the underlying LTS — no measure-theoretic reasoning is required.

2 List of the Protocols

2.1 Main Protocol: Asynchronous Byzantine Agreement (ABA)

(taken from [ABDY22]) Processes start with a bit $\{0, 1\}$ and may return a bit. a program implements ABA if it satisfies three properties: Validity, Agreement, Almost-Sure Termination. Validity: if a correct process returns b then a correct process had $1 - b$ as input. Agreement: if two correct processes return, then their output are equal. Termination: if $n - f$ correct processes take part to the protocol then, with probability 1, all correct processes will eventually return.

Protocol 1. Blablabla

Algorithm 1 Ideal Implementation of ABA

$x \leftarrow 0$
if $y > 0$ **then**
  loop
end if

Algorithm 2 Implementation of ABA taken from [ABDY22]

Protocol 2. **Input:** $b \in \{0, 1\}$

$r \leftarrow 1$
$decided \leftarrow \perp$
loop
  $(b, g) \leftarrow \text{GBCA}_r(b)$ ▷ $b \in \{0, 1, \perp\}$, $g \in \{A, B, C\}$
  $c \leftarrow \text{WCC}_r$ ▷ common coin for round r
  if $b = \perp$ **then**
    $b \leftarrow c$
  else if $g = A$ **then**
    $decided \leftarrow b$ ▷ keep participating to help others
  end if
  $r \leftarrow r + 1$
end loop
▷ TODO: verify termination rule

Definition 16. ABA.Concrete := ABA.Impl[GBCA.Impl, WCC.Impl], the instantiation of ABA.Impl with concrete implementations of the sub-protocols.

Definition 17. ABA.Hybrid := ABA.Impl[GBCA.Spec, WCC.Spec], the instantiation of ABA.Impl with ideal specifications of the sub-protocols.

2.2 ABA Sub-Protocols

Graded Binding Crusader Agreement (GBCA) (taken from [ABDY22, AW23]) Processes start with a binary input $\{0, 1\}$ and may return a value in $\{(0, A), (0, B), (\perp, C), (1, B), (1, A)\}$. A program implements GBCA if it satisfies four properties: Validity, Graded Agreement, Binding, Termination. Validity: if a correct process does not return (b, A) then a correct process had $1 - b$ as input. Graded Agreement: if two correct processes return (b, X) and (b', X') then $b = 0 \implies b' \neq 1$ and $X = A \implies X' \neq \perp$. Binding: when a correct process returns then a bound value b is defined and, in all extensions of the execution, correct processes do not return $(1 - b, B)$ or $(1 - b, A)$. Termination: if $n - f$ correct processes take part to the protocol then all correct processes will eventually return.

Protocol 3. Blablabla

Algorithm 3 Ideal Implementation of GBCA

$x \leftarrow 0$
if $y > 0$ **then**
  loop
end if

Protocol 4. Parameters: n processes with at most $f < n/3$ corrupted. Each process p_i has input $b_i \in \{0, 1\}$.

Algorithm 4 Implementation of GBCA taken from [ABDY22]

Input: $b \in \{0, 1\}$

send $\langle \text{INPUT}, b_i \rangle$ to all processes

upon receiving $\langle \text{INPUT}, b \rangle$ from at least $f + 1$ processes and not sent $\langle \text{INPUT}, b \rangle$ yet:
  send $\langle \text{INPUT}, b \rangle$ to all

upon receiving $\langle \text{INPUT}, b \rangle$ from at least $n - f$ processes:
  $Approved \leftarrow Approved \cup \{b\}$
  if not sent $\langle \text{ECHO}, \cdot \rangle$ yet:
    send $\langle \text{ECHO}, b \rangle$ to all

wait until
  (a) receiving $\langle \text{ECHO}, b \rangle$ from at least $n - f$ processes **or**
  (a) receiving $\langle \text{ECHO}, \cdot \rangle$ from at least $n - f$ processes and $|Approved| > 1$ **or**
To finish

Weak Common Coin (WCC) Processes start with no input and may return a bit. A program implements WCC if it satisfies three properties: ϵ-Goodness, Unpredictability, Termination. ϵ-Goodness: with probability ϵ, all correct processes return 0 (resp. 1). Unpredictability: Until $t + 1$ processes have called WCC, all outcomes are still possible. Termination: if $n - f$ correct processes take part to the protocol then all correct processes will eventually return.

Protocol 5. Blablabla

Algorithm 5 Ideal Implementation of WCC

$x \leftarrow 0$
if $y > 0$ **then**
  loop
end if

Protocol 6. Requires Gather and RSD

Algorithm 6 Implementation of WCC (yet to be determined)

$x \leftarrow 0$
if $y > 0$ **then**
  loop
end if

Definition 18. WCC.Concrete := WCC.Impl[Gather.Impl, RSD.Concrete], the instantiation of WCC.Impl with fully concrete sub-protocols.

Definition 19. WCC.Hybrid := WCC.Impl[Gather.Spec, RSD.Spec], the instantiation of WCC.Impl with ideal specifications of the sub-protocols.

2.3 WCC Sub-Protocols

Gather (taken from [SA21, SA24, AFW25]) Processes start with an input $x \in X$ and may return a partial function $f : Proc \rightharpoonup X$. A program implements Gather if it satisfies four properties: Validity, Agreement, Common Core, Termination. Validity: if a correct process returns f and $f(q)$ is defined and q is correct then $f(q)$ is q's input. Agreement: if two correct processes return f and g then $f(q)$ and $g(q)$ are equal or one of them is undefined. Common Core: at any point, there exists $S \subseteq Proc$ of size at least $n - f$ such that all f returned by a correct process is defined on S. Termination: if $n - f$ correct processes take part to Gather then all correct processes will eventually return.

Protocol 7. Blablabla

Algorithm 7 Ideal Implementation of Gather

```
x ← 0
if y > 0 then
    loop
end if
```

Protocol 8. Might require BRB

Algorithm 8 Implementation of Gather (yet to be determined)

```
x ← 0
if y > 0 then
    loop
end if
```

Random Secret Draw (RSD)

Protocol 9. Blablabla

Algorithm 9 Ideal Implementation of RSD (yet to be determined)

```
x ← 0
if y > 0 then
    loop
end if
```

Protocol 10. Require AVSS and BRB

Definition 20. RSD.Concrete := RSD.Impl[BRB.Impl, AVSS.Spec], the instantiation of RSD.Impl with the concrete implementation of BRB. AVSS is used as a black-box, so its specification is plugged in directly.

Definition 21. RSD.Hybrid := RSD.Impl[BRB.Spec, AVSS.Spec], the instantiation of RSD.Impl with the ideal specifications of BRB and AVSS.

Algorithm 10 Implementation of RSD (yet to be determined)

$x \leftarrow 0$
if $y > 0$ **then**
  loop
end if

2.4 Basic Primitives

Byzantine Reliable Broadcast (BRB) (taken from [Bra87]) Each instance of BRB has a designated *leader* with an input message m and processes may return a received message. A program implements BRB if it satisfies three properties: Validity, Totality and Agreement. Validity: if the leader is not corrupted then all correct processes must return its input. Agreement: if two correct processes return then they return the same value. Totality: if a correct process returns then all correct processes eventually return.

Protocol 11. Blablabla

Algorithm 11 Ideal Implementation of BRB

$x \leftarrow 0$
if $y > 0$ **then**
  loop
end if

Protocol 12. Requires no sub-protocol. Parameters: n processes with at most $f < n/3$ corrupted; leader ℓ with input m.

Algorithm 12 Implementation of BRB taken from [Bra87]

Code for the leader ℓ with input m:
  send $\langle \text{INIT}, m \rangle$ to all processes

Code for every process p_i:
upon first reception of $\langle \text{INIT}, m \rangle$ from ℓ **do**
  send $\langle \text{ECHO}, m \rangle$ to all processes
upon reception of $\langle \text{ECHO}, m \rangle$ from at least $\lceil (n + f + 1)/2 \rceil$ distinct processes **do**
  if $\langle \text{READY}, \cdot \rangle$ has not yet been sent **then**
    send $\langle \text{READY}, m \rangle$ to all processes
upon reception of $\langle \text{READY}, m \rangle$ from at least $f + 1$ distinct processes **do**
  if $\langle \text{READY}, \cdot \rangle$ has not yet been sent **then**
    send $\langle \text{READY}, m \rangle$ to all processes
upon reception of $\langle \text{READY}, m \rangle$ from at least $2f + 1$ distinct processes **do**
  deliver m

Asynchronous Verifiable Secret Sharing (AVSS) TO IMPROVE (taken from [CR93]) AVSS is actually a pair of protocol (Sh, Rec) with a designated *dealer* owning a secret s. A program implements a $(1 - \epsilon)$-correct AVSS if it satisfies six properties: Valid Termination, Total Termination, *Rec* Termination, Binding, Secrecy. Valid Termination: if the dealer is honest then all correct processes will eventually terminate Sh. Total Termination: if a correct

	Linear Safety	Branching Safety	Liveness
ABA	Validity & Agreement		
GBCA	Validity & Agreement	Binding	Termination
WCC			Termination
Gather	Validity & Agreement & Common Core		Termination
RSD (???)			
BRB	Validity & Agreement		Totality
AVSS (???)			

Table 1: Table of CTL Properties

	Probabilistic Safety	Probabilistic Liveness	Secrecy
ABA		Almost-Sure Termination	
GBCA			
WCC	ϵ-Goodness		Unpredictability
Gather			
RSD (???)			
BRB			
AVSS (???)			

Table 2: Table of non-CTL Properties

process terminates Sh then all correct processes will eventually terminate Sh. Rec Termination: if one correct process has terminated Sh and $n - f$ correct processes take part to Rec then all correct processes will eventually terminate Rec. Binding: when a correct process has completed Sh, a *shared secret* s is defined and, with probability $1 - \epsilon$, (a) if the dealer is honest then s is its secret input and (b), in all extensions of the execution, the Rec return value of correct processes is s. Secrecy: if the dealer is honest and no correct process started Rec then the adversary have no information about the shared secret.

Protocol 13. Blablabla

Algorithm 13 AVSS Specification

```
x ← 0
if y > 0 then
    loop
end if
```

AVSS is used as a blackbox so no implementation.

2.5 Summary of Properties

3 Proof Decomposition

The end goal is the correctness of ABA.Impl. We establish it via a simulation by ABA.Spec and transport the spec's correctness. The simulation itself is decomposed recursively along the sub-protocol structure.

For a protocol P with concrete sub-protocols $Q_1, \ldots, Q_k$, the step "P.Spec simulates P.Impl" is split into:

1. (Substitution) $\mathsf{P.Impl}[\mathsf{Q}_1.\mathsf{Spec}, \ldots, \mathsf{Q}_k.\mathsf{Spec}]$ is simulated by $\mathsf{P.Impl}$, using the child simulations $\mathsf{Q}_i.\mathsf{Spec}$ simulates $\mathsf{Q}_i.\mathsf{Impl}$ and compositionality.
2. (Core) $\mathsf{P.Spec}$ simulates the hybrid $\mathsf{P.Impl}[\mathsf{Q}_1.\mathsf{Spec}, \ldots, \mathsf{Q}_k.\mathsf{Spec}]$.
3. (Transitivity) Compose (1) and (2).

Leaves of the recursion (no sub-protocol, or only the black-box $\mathsf{AVSS.Spec}$) skip the substitution step.

3.1 Top-level

Theorem 22. $\mathsf{ABA.Concrete}$ *satisfies Validity, Agreement and Almost-Sure Termination.*

Proof. Transport the corresponding properties of $\mathsf{ABA.Spec}$ along the simulation 23. □

3.2 ABA

Theorem 23. $\mathsf{ABA.Spec}$ *simulates* $\mathsf{ABA.Concrete}$.

Proof. Transitivity of simulation applied to 24 and 25. □

Lemma 24. $\mathsf{ABA.Hybrid}$ *is simulated by* $\mathsf{ABA.Concrete}$.

Proof. Compositionality of simulation applied to 29 and 26. □

Lemma 25. $\mathsf{ABA.Spec}$ *simulates* $\mathsf{ABA.Hybrid}$.

3.3 WCC

Theorem 26. $\mathsf{WCC.Spec}$ *simulates* $\mathsf{WCC.Concrete}$.

Proof. Transitivity of simulation applied to 27 and 28. □

Lemma 27. $\mathsf{WCC.Hybrid}$ *is simulated by* $\mathsf{WCC.Concrete}$.

Proof. Compositionality of simulation applied to 30 and 31. □

Lemma 28. $\mathsf{WCC.Spec}$ *simulates* $\mathsf{WCC.Hybrid}$.

3.4 GBCA

Theorem 29. $\mathsf{GBCA.Spec}$ *simulates* $\mathsf{GBCA.Impl}$. *No sub-protocol: direct proof.*

3.5 Gather

Theorem 30. $\mathsf{Gather.Spec}$ *simulates* $\mathsf{Gather.Impl}$. *Sub-protocol structure TBD.*

3.6 RSD

Theorem 31. RSD.Spec *simulates* RSD.Concrete.

Proof. Transitivity of simulation applied to 32 and 33. □

Lemma 32. RSD.Hybrid *is simulated by* RSD.Concrete. *Only BRB is substituted;* AVSS.Spec *stays fixed on both sides.*

Proof. Compositionality of simulation applied to 34. □

Lemma 33. RSD.Spec *simulates* RSD.Hybrid.

3.7 BRB

Theorem 34. BRB.Spec *simulates* BRB.Impl. *No sub-protocol: direct proof.*

Bibliography

[ABDY22] Ittai Abraham, Naama Ben-David, and Sravya Yandamuri. Efficient and adaptively secure asynchronous binary agreement via binding crusader agreement. In *41st ACM Symposium on Principles of Distributed Computing*, pages 381–391, 2022.

[AFW25] Hagit Attiya, Itay Flam, and Jennifer L. Welch. Beyond canonical rounds: Communication abstractions for optimal byzantine resilience. *CoRR*, abs/2510.04310, 2025.

[AW23] Hagit Attiya and Jennifer L. Welch. Multi-valued connected consensus: A new perspective on crusader agreement and adopt-commit. In *27th International Conference on Principles of Distributed Systems (OPODIS)*, 2023.

[Bra87] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.

[CR93] Ran Canetti and Tal Rabin. Fast asynchronous byzantine agreement with optimal resilience. In *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing*, STOC '93, page 42–51, New York, NY, USA, 1993. Association for Computing Machinery.

[SA21] Gilad Stern and Ittai Abraham. Living with asynchrony: the gather protocol. https://decentralizedthoughts.github.io/2021-03-26-living-with-asynchrony-the-gather-protocol, 2021.

[SA24] Gilad Stern and Ittai Abraham. Gather with binding and verifiability. https://decentralizedthoughts.github.io/2024-01-09-gather-with-binding-and-verifiability/, 2024.